

Content Security Policy (CSP) Effectiveness Evaluation

Project submitted to the
SRM University – AP, Andhra Pradesh
for the partial fulfilment of the requirements to award the degree of

Bachelor of Technology
In
Computer Science and Engineering
School of Engineering and Sciences

Submitted by
Bhavajna Madivada
(AP22110011280)



Under the Guidance of
Dr. Bhaskar Santhosh E

SRM University–AP
Neerukonda, Mangalagiri, Guntur
Andhra Pradesh – 522 240
[November, 2025]

1. Problem Statement

Web applications today face continuous threats from client-side attacks, especially **Cross-Site Scripting (XSS)**.

XSS remains one of the most prevalent vulnerabilities because even a small oversight such as:

- a missing input sanitization,
- an unsafe DOM manipulation,
- or an improperly handled user input

can allow attackers to inject and execute malicious JavaScript directly in the victim's browser.

To mitigate such attacks, the industry uses **Content Security Policy (CSP)** — a browser-enforced security mechanism that allows developers to specify which resources (scripts, images, styles) are allowed to load.

1.1 What is Content Security Policy (CSP)?

Content Security Policy (CSP) is a **browser-enforced security standard** that restricts the sources from which scripts, images, frames, styles, and other resources can load. It acts as an **allow-list**: only resources explicitly permitted by the developer are allowed. All other content is blocked by the browser.

A CSP policy is delivered via:

- The **HTTP response header**
 - Content-Security-Policy: <rules>
- Or an HTML **meta tag**

The browser interprets these CSP rules and prevents execution of unauthorized scripts. This makes CSP an essential mitigation technique against XSS, clickjacking, data injection, malicious resource loading, and HTML-based code injection.

Where is CSP Implemented?

CSP is implemented directly on the server side and delivered to the browser through:

- Web server configuration (Apache, Nginx, Node.js)
- Application code (PHP `header()`, Express.js `res.setHeader()`, etc.)
- Meta tags for limited enforcement

In this project, CSP is applied at the **PHP level** using the `header()` function inside each test page.

CSP is intended to act as a **powerful second line of defense**, even if the application itself mistakenly exposes an XSS entry point.

However, a critical challenge arises:

CSP only protects effectively when it is configured correctly.

Developers often deploy CSP headers assuming they are secure, but small syntax errors, missing semicolons, incorrect values, or overly broad allow-lists can silently weaken or completely disable CSP.

Therefore, this project focuses on **testing whether a deployed CSP is genuinely effective**, identifying misconfigurations, demonstrating how attackers can exploit incorrect CSP rules, and verifying how strict CSP mitigates real-world XSS attacks.

2. Target Environment

Component	Details
Local Server	XAMPP (Apache + PHP)
Testing URL	http://localhost/vuln-lab/modules/
Pages Tested	home.php, reflected.php, stored.php, dom.php
CSP File Used	Applied in each PHP page via stored in csp_script.php
Tools Used	Chrome Developer Tools, Local PHP Scripts
XSS Payloads Tested	<script>alert(1)</script>, image onerror payloads

File Structure (as per project)

- index.php – Main navigation
- home.php – Demo page
- reflected.php – Reflected XSS
- stored.php – Stored XSS (comments saved)
- dom.php – DOM-based XSS using innerHTML
- csp_strict.php – Strict CSP enforced

- comments.txt — Stores payloads for Stored XSS
- style.css — UI styling

3. Methodology

The project followed a structured that aligned with OWASP XSS testing guidelines.

3.1 Reconnaissance

- Identified vulnerable pages for Reflected, Stored, and DOM XSS.
- Inspected HTML source to identify input reflection points.
- Verified absence of sanitization mechanisms.

3.2 Enumeration

- Tested multiple payloads to identify:
 - Inline script execution
 - Event-handler-based XSS (onerror, onload)
 - DOM-based reflection vulnerabilities

3.3 CSP Application

- Applied the restrictive CSP on all pages:
default-src 'self';
script-src 'self';
img-src 'self';
object-src 'none';
base-uri 'self';
- Effects:
 - Blocks inline scripts
 - Blocks external malicious scripts
 - Blocks event-handler XSS
 - Restricts images to local domain

3.4 Exploitation

- Executed XSS payloads in two conditions:
- Without CSP → Payload executed normally.
- With Strict CSP → Browser blocked execution and logged CSP violation reports.

4. Findings

4.1 Reflected XSS (reflected.php)

Description

User input is taken from \$_GET and reflected directly into HTML without sanitization.

Payload Used

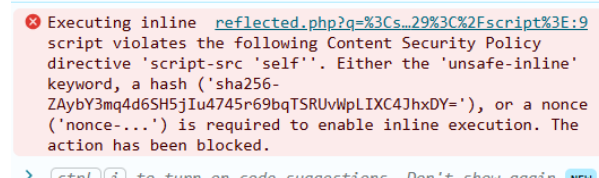
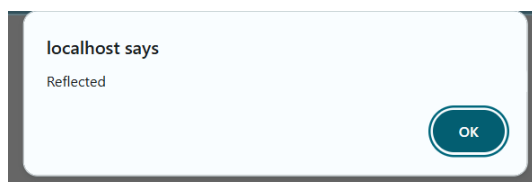
```
<script>alert('Reflected')</script>
```

Without CSP

- Script executed.
- Alert pop-up shown.

With Strict CSP

- Script did NOT execute.
- Browser console logged CSP violation.
- Inline scripts were fully blocked.



Risk Level: High (when CSP not used)

4.2 Stored XSS (stored.php)

Description

User comments stored in comments.txt and re-displayed on the page.

Payload

```
<script>alert('Stored')</script>
```

Without CSP

- Script executed when page loaded again.

- Persistent XSS achieved.

With Strict CSP

- Inline script blocked from executing.
- Stored XSS attempts unsuccessful.



✖ Executing inline script violates the following Content Security Policy directive 'script-src 'self''. Either the 'unsafe-inline' keyword, a hash ('sha256-/ZAedbwrJN1bArRZ+SiGQoduuEGwTFeXONTYYzjaxY='), or a nonce ('nonce-...') is required to enable inline execution. The action has been blocked. [stored.php:13](#)

Risk Level: Critical (without CSP)

4.3 DOM-Based XSS (dom.php)

Description

The page reads location.search and injects it using:

```
element.innerHTML = userInput;
```

Payload

```
dom.php?msg=<img src=x.png onerror=alert(1)>
```

Without CSP

- DOM-based script executed.

With Strict CSP

- Inline JavaScript blocked.
- DOM-based injection prevented.

DOM XSS Test



Risk Level: High (without CSP)

✖ Executing inline event handler violates the following Content Security Policy directive 'script-src 'self''. Either the 'unsafe-inline' keyword, a hash ('sha256-...'), or a nonce ('nonce-...') is required to enable inline execution. Note that hashes do not apply to event handlers, style attributes and javascript: navigations unless the 'unsafe-hashes' keyword is present. The action has been blocked. [dom.php:10](#)

4.4 Home Page Attack Scenario (home.php)

Payload

```
<script>alert(1)</script>
```

Finding

- Without CSP → Executes.
- With strict CSP → Blocked.



```
Executing inline home.php?x=%3Cscript%3E%3C%2Fscript%3E:18
script violates the following Content Security Policy
directive 'script-src 'self'''. Either the 'unsafe-inline'
keyword, a hash ('sha256-
bhHHL3z2vDgxUt0W3dWQOrprscmda2Y5pLsLg4GF+pI='), or a nonce
('nonce-...') is required to enable inline execution. The
action has been blocked.
```

5. Common Developer Mistakes in CSP Configuration

During the research phase, several frequently occurring mistakes were identified. These mistakes may appear small but have **major security consequences**.

5.1 Mistake 1 — Incorrect Protocol (Missing Colon)

Incorrect:

Content-Security-Policy: script-src 'self' https

Correct:

Content-Security-Policy: script-src 'self' https:

Without the colon, browsers interpret https as a *domain*, not a protocol. This effectively allows **all HTTPS domains** to load scripts, enabling attackers to execute malicious code from any secure website.

5.2 Mistake 2 — Missing Semicolons Between Directives

Incorrect:

frame-ancestors 'self' https://example.com default-src 'none'

Correct:

frame-ancestors 'self' https://example.com; default-src 'none';

A missing semicolon merges two directives into one invalid line. As a result:

- frame-ancestors breaks,
- default-src is ignored,

- and clickjacking or script injection protections fail.

5.3 Mistake 3 — Incorrect or Unquoted Special Values

Incorrect:

Content-Security-Policy: frame-ancestors DENY

There is no DENY keyword in CSP.

Correct:

Content-Security-Policy: frame-ancestors 'none'

Special tokens like 'self', 'none', and 'unsafe-inline' must always be quoted.

5.4 Mistake 4 — Unquoted Hashes or Nonces

Incorrect:

Content-Security-Policy: script-src sha512-BASE64HASH

Correct:

Content-Security-Policy: script-src 'sha512-BASE64HASH'

If hashes or nonces are unquoted, CSP **ignores** them, enabling inline script execution — completely defeating the main goal of CSP.

6. Comparisons

Effectiveness of CSP Across All Attack Types

Attack Type	Without CSP	With Strict CSP	Result
Reflected XSS	Executed	Blocked	CSP effective
Stored XSS	Executed	Blocked	CSP effective
DOM XSS	Executed	Blocked	CSP effective
Inline Scripts	Allowed	Blocked	CSP effective
Image onerror Payloads	Allowed	Blocked (img-src 'self')	CSP effective

Attack Type	Without CSP	With Strict CSP	Result
External Script Injections	Allowed	Blocked (script-src 'self')	CSP effective

7. Recommended Security Measures

Based on OWASP standards and my practical results, the fix is:

Use a **Strict CSP** such as:

- script-src 'self'
- No unsafe-inline
- No unsafe-eval
- No wildcard sources
- No unnecessary external domains
- object-src 'none'
- Strict img-src, connect-src, and base-uri
- Use nonces or script hashes where needed

8. Conclusion

The evaluation clearly demonstrates that **Strict CSP provides significant protection against all forms of XSS tested**, including:

- Reflected XSS
- Stored XSS
- DOM-based XSS
- Inline script execution
- Image-based event handler attacks

Key takeaways:

- Without CSP → every XSS payload executed successfully.
- With proper CSP → **all tested payloads were blocked**.
- CSP acts as a **strong browser-level security layer**, but:
 - Developers must still sanitize inputs.
 - CSP should not be the only line of defense.
 - Secure coding + CSP = optimal protection.

9. References

- [OWASP Web Security Testing Guide](#)
- [Mozilla Developer Network \(MDN\) – Content Security Policy](#)
- [Google Web Fundamentals – CSP Best Practices](#)

Github Link: <https://github.com/BhavajnaMadivada/CSP-Effectiveness-Testing>